Evolutionary vs imitation learning for neuromorphic control at the edge*

NEUROMORPHIC
Computing and Engineering

PAPER

Evolutionary vs imitation learning for neuromorphic control at the edge*

Catherine Schuman[1,2,**], Robert Patton[1], Shruti Kulkarni[1], Maryam Parsa[3], Christopher Stahl[1], N Quentin Haas[1], J Parker Mitchell[1], Shay Snyder[4], Amelie Nagle[5], Alexandra Shanafield[5] and Thomas Potok[1]

[1] Oak Ridge National Laboratory, Oak Ridge, TN, United States of America
[2] University of Tennessee, Knoxville, TN, University of Tennessee, Knoxville, TN, United States of America
[3] George Mason University, Fairfax, VA, United States of America
[4] East Tennessee State University, Johnson City, TN, United States of America
[5] Oak Ridge High School, Oak Ridge TN, United States of America
** Author to whom any correspondence should be addressed.

E-mail: cshuman@utk.edu

Keywords: evolutionary optimization, spiking neural networks, imitation learning, autonomous driving

OPEN ACCESS

RECEIVED
6 October 2021

REVISED
17 December 2021

ACCEPTED FOR PUBLICATION
22 December 2021

PUBLISHED
24 January 2022

Abstract

Neuromorphic computing offers the opportunity to implement extremely low power artificial intelligence at the edge. Control applications, such as autonomous vehicles and robotics, are also of great interest for neuromorphic systems at the edge. It is not clear, however, what the best neuromorphic training approaches are for control applications at the edge. In this work, we implement and compare the performance of evolutionary optimization and imitation learning approaches on an autonomous race car control task using an edge neuromorphic implementation. We show that the evolutionary approaches tend to achieve better performing smaller network sizes that are well-suited to edge deployment, but they also take significantly longer to train. We also describe a workflow to allow for future algorithmic comparisons for neuromorphic hardware on control applications at the edge.

1. Introduction

Intelligent computation at the edge is increasingly prevalent with the rise in popularity of 'smart' systems, from smart homes to smart transportation to the smart grid [9, 16]. Performing intelligent operation at the edge requires computing platforms that are capable of artificial intelligence computations under strict size, weight, and power constraints. As such, custom hardware systems are increasingly popular for performing these computations at the edge. In addition to application specific hardware, as well as custom hardware systems for performing conventional artificial intelligence (AI) operations at the edge, such as the NVIDIA Jetson Nano [8], neuromorphic computers are an attractive technology to implement edge AI.

Neuromorphic computers are computers where both the structure (the architecture) and the operation of the computer is inspired by biological brains [14, 24, 44]. Neuromorphic computers have several properties that make them attractive for edge computing applications, primarily low power operation, robustness and resilience, and adaptability and plasticity. There have been several compelling demonstrations of neuromorphic computers for edge applications, including for keyword spotting [7], robotics [3, 25, 35, 49],

medical applications [5], and intelligent engine control [45]. However, there are many different options for neuromorphic training algorithms, and it is not always clear what the best algorithm is for a given application.

A great potential application for neuromorphic computers at the edge is autonomous vehicles. The computation required to control autonomous vehicles is tremendous, with compute dominating from 40 to 80 percent of the power consumption required for the autonomous control system (including sensors) [6, 18]. If the power consumption requirements of autonomous vehicles can be reduced using, for example, neuromorphic computers, this will have an impact on the battery usage of future autonomous, electric vehicles. Neuromorphic platforms designed with memristive devices have also been demonstrated to perform well for autonomous navigation tasks, offering benefits in energy efficiency and reduced latency. Wang *et al*, demonstrate a fully analog hardware for controlling an autonomous vehicle which incorporated online learning through modification of memristive conductance through supervised learning with a response time in the order of few tens of nanoseconds [57]. Other applications of neuromorphic memristive hardware have also been in robot control systems, such as mobile inverted pendulum using traditional control algorithms [10]. In this work, the hybrid analog-digital platform realized Kalman filters and proportional derivative control algorithms for sensor fusion and motion control tasks, respectively. In this work, we take a step towards evaluating neuromorphic solutions for autonomous vehicles by specifically examining performance for a small-scale autonomous race car. The platform we use is the F1Tenth system[6], a one-tenth scale Formula One racing vehicle. In addition to a physical vehicle, there is also an F1Tenth simulator [4] that can be used for training purposes.

In this work, we evaluate the feasibility of using neuromorphic computing for this small-scale autonomous racing vehicle. We evaluate two evolutionary approaches as well as two imitation learning approaches, for training spiking neural networks for neuromorphic deployment for this task. We describe the advantages and disadvantages of each of these approaches, specifically in the context of edge deployment of neuromorphic systems. Our key contributions in this work are:

- An evaluation of imitation learning and evolutionary learning approaches for an autonomous driving application
- A comparison of the performance of different training approaches in the context of edge deployment
- A complete workflow for evaluating training algorithms for control that can be easily extended to more control applications and hardware implementations in the future
- A demonstration of spiking neural networks on a physical autonomous vehicle

We show that the evolutionary approaches tended to outperform the imitation learning approaches in terms of accuracy, and we also show that the evolutionary approaches tended to produce smaller networks that are more well-suited for edge deployment than those trained with imitation learning.

2. Background and related work

The traditional approaches in designing controllers for autonomous systems (cars' speed control/navigation, or robots, or plant controllers, etc) have used the PID (proportional–integral–derivative) control, where the system's response is directed towards the target based on its observed error. Most of the research in designing control systems is focused on finding different ways of tuning the parameters of the PID controller [15, 59]. Other control approaches in the application of autonomous driving make use of physics based models that account for the vehicle dynamics and predictive control methods such as Bayes learning [15, 23]. Specifically for the F1TENTH competition, Sinha *et al*, have shown methods to achieve robustness by generating diverse sets of opponents and incorporating reinforcement learning with robust bandit optimization approach to achieve both speed in the autonomous vehicle and avoid crashing [50]. Reinforcement learning has also been shown as an effective way to implement online learning of autonomous driving with collision avoidance [26].

There have been a variety of neuromorphic approaches for control tasks that have been presented in the literature. A common approach for robotic tasks such as gait generation are to implement central pattern generators onto a neuromorphic system [20, 38, 51], which have a fixed structure for a given task, transitioning between different states. These approaches are often hand-tooled for a given neuromorphic implementation, and it is not clear what the appropriate central pattern generator would be for each individual task. There have been multiple neuromorphic solutions for designing the PID control [52, 62] or proportional integral control system [19, 61]. Again, these are often hand-tooled for a given solution. Stagsted *et al*, have demonstrated a neuromorphic PID controller for UAVs, implemented on the Loihi platform [52]. Their implementation does not incorporate learning within the network, and the performance of the network depends on the number of neurons. There have been some early neuromorphic works in realizing PID controllers with multi-layer

[6] URL: https://f1tenth.org/.

neural networks (with a pre-defined neural network structure) trained with backpropagation which tuned the parameters of the PID controller [1]. This approach was employed in performing simple tasks such as temperature control. Some more recent neuromorphic solutions to PID controllers for robotic control have been implemented in the neural engineering framework [53, 60]. This neuromorphic PID controller required close to 750 spiking neurons to realize all the different paths, and about 250 for the inverse kinematics (IK) of the robot, and the performance of the system was dependent on the number of neurons. The parameters of the PID and IK were trained using the prescribed error sensitivity rule. Other neuromorphic PID control demonstrations have been shown in LIDAR guided autonomous car, specifically with focus on collision avoidance [48]. In their approach as well, the system's performance improves with increasing number of neurons. In this work, we focus on approaches that are more broadly applicable and do not require significant hand-tuning (or pre-defining the network structure).

We focus on two approaches for training neuromorphic networks for control: evolutionary approaches and imitation learning. Genetic or evolutionary approaches have been commonly used to produce neuromorphic solutions to a variety of control tasks, including robotic control [2, 28], drone control [21], video games [37], and engine control [45]. Imitation learning has also been popularly used for control of neuromorphic systems, especially for self-driving robots [17, 22]. These cases typically implement a convolutional neural network to map observations to actions.

In this work we do not investigate any reinforcement learning approaches for neuromorphic training, though those have been used in the literature for tasks such as the Pong video game [58], grid world tasks [39], short-term trajectory planning [27], and autonomous robot navigation [3]. In the future, we hope to add reinforcement learning as a comparison point.

Finally, it is worth noting that neuromorphic systems have also been used for sensing applications in autonomous vehicles, especially for event-based sensors [11, 35, 56]. There is tremendous opportunity for neuromorphic systems to perform efficient, native processing of sensors such as LIDAR and dynamic vision or event-based sensors, but in this work, we also hope to showcase that neuromorphic systems can be part of the control system as well.

3. Method

In this work, we evaluate a variety of training approaches for neuromorphic systems at the edge for a particular autonomous racing application. In this section, we describe the components of our approach, including the details of the application environment, the hardware implementation used, the system software used to connect all of the components, and the four separate training approaches. The complete workflow for these methods and how they are connected is shown in figure 1.

3.1. Application: F1TENTH

The application and evaluation platform used in this work comes from the F1TENTH community. The F1TENTH community provides a suite of resources for a 1/10th scale Formula One competition, including specifications for the physical car, instructions for assembling and running the hardware and software, software for interacting with the car, and simulation software. In this work, we use both the F1TENTH Open AI gym environment for training and testing, as well as the physical F1TENTH car for real-world evaluation. The physical car and its components are shown in figure 2.

For training purposes, we use the F1TENTH Open AI gym simulator [31]. Included with the simulators are 1/10th scale Formula One racetracks. Here, we use five of the real-world tracks for training and fifteen for testing. The training tracks are shown in figure 3, and the testing tracks are listed in the right y-axis of figure 7.

To match the physical platform's LIDAR sensor's specifications, we update the number of LIDAR beams per scan cycle in the simulation to 1440 and the field of view to 6.283 1853 rad. The LIDAR we have used on the physical platform provides a 270 degree front-facing view of the environment. Because the simulator provides a 360 LIDAR view, we then down-sample the simulated LIDAR to the 960 beams we expect from the physical LIDAR by taking beams 240 through 1200 in from the 1440 provided by the simulator. Finally, to reduce the input to the neuromorphic implementation, we further down-sample the LIDAR readings by providing a total of ten beams to the neuromorphic implementations, where the 960 beams are divided into ten equal-sized contiguous regions and the maximum value in each of those regions is passed as input to the neuromorphic implementation. Additionally on the physical vehicle, the LIDAR are presented counterclockwise, and in the simulation the LIDAR are presented clockwise, so we reverse the order in the physical environment before pre-processing the data.

Over the course of operation, the simulator provides state information about how the vehicle is performing as driving around the track. The simulator provides the LIDAR sensor information as described above, which

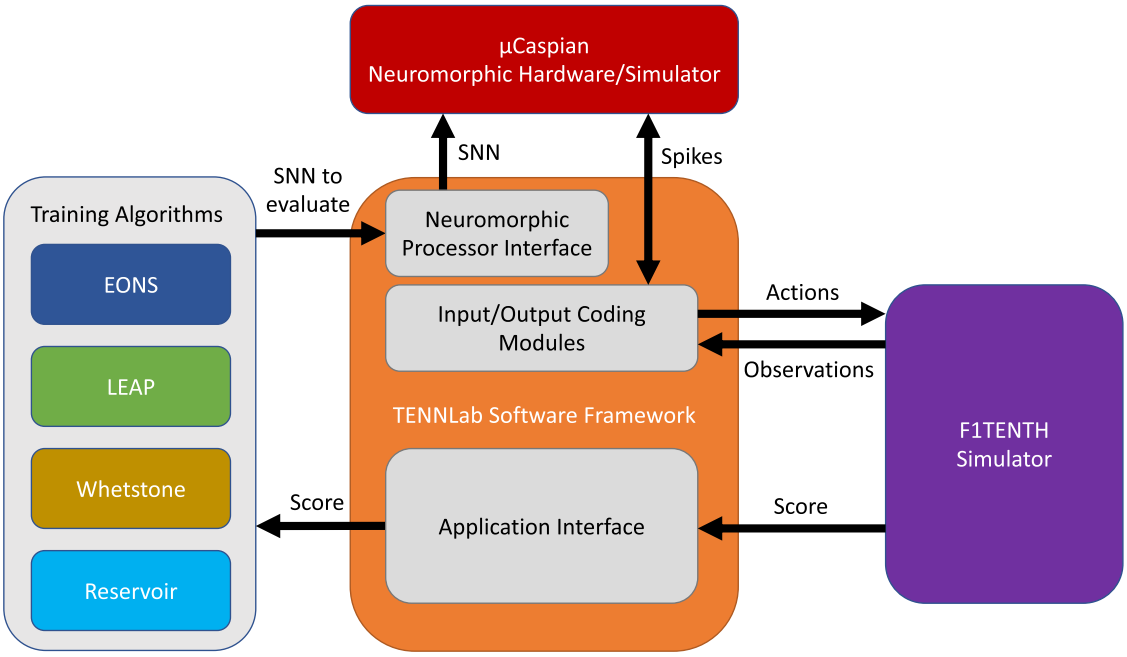


Figure 1. Complete workflow for this approach, with a hardware/simulator component, training approaches, and the application, as well as the system software that connects all of the individual components.

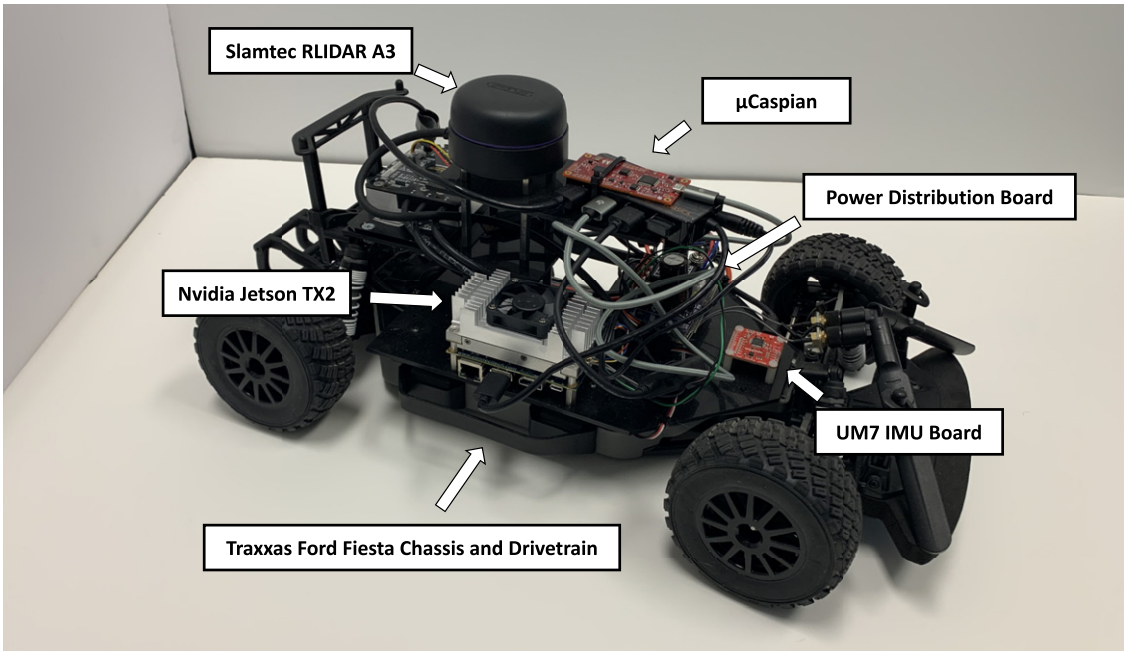


Figure 2. F1TENTH physical evaluation platform and the associated components used for real-world evaluation.

we use as input to the spiking neural network. The simulator also provides information about the distance traveled, collisions, and laps completed, which we use as part of our fitness evaluations in some of the algorithms. The actions that can be applied to the simulator are speed and steering angle every 5 milliseconds. As such, the spiking neural network (SNN) will receive ten LIDAR beams as input and needs to produce a speed and steering angle as output in less than 5 milliseconds to operate in real time. The approach for encoding the data into spikes and then post-processing the spikes from the neuromorphic implementation back into speed and steering angle values depends on the neuromorphic algorithm used; we describe those in more detail in section 3.4. Rather than producing continuous output for the steering and speed values, we instead allow for 29 possible steering angles and 11 possible speed values, shown in table 1. We selected the maximum and minimum values for the steering angle to be consistent with what is feasible on the physical platform. In this work,

Figure 3. Formula One tracks used for training with the F1TENTH simulator [31].

Table 1. Outputs and values.

Output type	Values
Steering angle (radians where 0 is pointing forward)	0, −0.01, 0.01, −0.03, 0.03, −0.05, 0.05, −0.07, 0.07, −0.1, 0.1, −0.13, 0.13, −0.15, 0.15, −0.17, 0.17, −0.2, 0.2, −0.23, 0.23, −0.25, 0.25, −0.27, 0.27, −0.3, 0.3, −0.34, 0.34
Speed (m s^{-1})	1, 1.1, 1.2, 1.3, 1.4, 1.5, 1.6, 1.7, 1.8, 1.9, 2

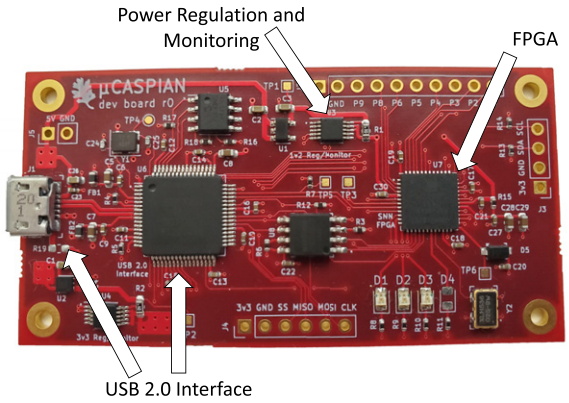


Figure 4. μCaspian development board. The full system specifications are described in [30].

we focus on completing laps and avoiding collisions, so we do not reward speed. Instead, we selected speed values that would minimize damage due to collisions in the physical environment. The simulator has two stopping conditions for a given track; either the car crashes into a wall (collision) or the car successfully crosses the starting line twice. However, the simulator does not enforce traveling in clockwise or counterclockwise on the track. It registers 'laps completed' as times the vehicle crosses the starting line, which may not require a full completion of a lap. Thus, any evaluation metric must take this into account.

3.2. Hardware implementation: μCaspian

We targeted μCaspian as our hardware implementation [30]. The broader Caspian neuromorphic development platform includes neuromorphic hardware implementations on field programmable gate arrays (FPGAs) for different sized FPGAs, system software, and hardware-accurate software simulators [29]. The μCaspian implementation is specifically targeted towards edge applications (figure 4). The μCaspian architecture is implemented on a very small low cost FPGA, the Lattice iCE40 UP5K1. The full development board also includes options for both USB communication and direct I/O interfaces. When using the USB communication on the development board, the μCaspian board consumes significantly more power, at about 500 mW. Without the USB communication, the FPGA alone consumes about 10–20 mW. As such, if power constraints are a significant concern, the direct I/O interfaces are an option, but for ease of use, the USB interface is also available. The μCaspian board can realize spiking neural networks of up to 256 neurons and 4096 synapses. However, because μCaspian is part of a family of architectures, if larger networks are required, larger, more expensive, more power-intensive FPGAs can be used.

μCaspian implements leaky integrate and fire neurons with axonal delay, but in this work, to simplify the parameters to optimize and to use the same neuron model across all algorithms, we turn off both leak

and axonal delay, so the neurons are simply integrate and fire neurons with a single threshold parameter. μCaspian synapses have both weights and synaptic delays, and we use both of those as parameters in our training approaches. All Caspian parameters are integers. Here, we allow neuron threshold values to be between 0 and 255, synaptic weight values to be between -255 and 255, and synaptic delay values to be between 0 and 15.

3.3. System software: TENNLab

To communicate with the μCaspian system and simulation, interface with the F1TENTH OpenAI gym implementation, and connect with the training algorithms described in section 3.4, we use the TENNLab neuromorphic computing software framework [37]. The TENNLab framework provides a common interface to multiple neuromorphic hardware and simulator backends, including Caspian. The TENNLab framework has both C++ and Python interfaces, and in this work, we use the Python framework, which comes equipped with an interface to train spiking neural networks for OpenAI gym environments. TENNLab also implements a variety of input encoding [41] and output decoding approaches. The TENNLab framework includes a variety of algorithmic approaches for training spiking neural networks, including evolutionary approaches, reservoir computing, Whetstone and decision tree [43].

3.4. Training algorithms

In this section, we describe the two training approaches and four training algorithms for control tasks that we use in this work. We focused our attention on evolutionary optimization because of its proven success on neuromorphic control applications [36] and imitation learning because of its broad use in autonomous driving applications. In the future, we also plan to investigate other approaches for neuromorphic algorithms, including reinforcement learning.

3.4.1. Evolutionary training

For evolutionary training approaches, we must define a fitness function to evaluate a network. There are several behaviors that we want to encourage, besides completing two laps without colliding with a wall:

- Driving mostly straight, penalizing non-zero steering angles.
- Performing any driving maneuvers that substantially increases the distance traveled over what would be expected for the track (e.g., U-turns on the track).

Thus, we define our fitness function as follows, where net is the network to be evaluated and tr is the track:

$$f(\text{net}, \text{tr}) = \begin{cases} \dfrac{d}{D_{\text{tr}}} & L < 2 \\ -1 & L = 2 \;\;\&\&\;\; d - D_{\text{track}} > 10 \\ \dfrac{d}{D_{\text{tr}}} + \dfrac{1}{\sum_{i=0}^{T} \alpha_i} & \text{otherwise} \end{cases} \tag{1}$$

where d is the total distance traveled, D_{tr} is the nominal length to travel two laps for that track, α_i is the steering angle for time step i, and T is the total number of successful time steps completed without colliding, and L is the total laps completed. The goal of this fitness function is to encourage covering as much distance as possible until two laps are completed. Once the networks can complete two laps for that track, if the distance is substantially more than the nominal distance, it is penalized significantly by setting the fitness score to -1. If two laps are completed and the distance is not substantially more, the fitness function then rewards smaller steering angles used throughout $\left(\frac{1}{\sum_{i=0}^{T}\alpha_i}\right)$. This fitness evaluation was determined experimentally; in the future, we plan to explore different fitness evaluation approaches to encourage different behaviors. The full fitness evaluation as the average across five training tracks. We originated a version of this fitness function in [33].

We use two evolutionary approaches for training spiking neural networks. Both evolutionary approaches use the same fitness evaluation as described above. For each network that is evaluated, the network is loaded onto the μCaspian simulator and then evaluated on each of the five training tracks shown in figure 3. The workflow of a general evolutionary algorithm is shown in figure 5, but the specifics are customized to the particular algorithm.

The first evolutionary approach, evolutionary optimization for neuromorphic systems (EONS) [40, 42] is specifically used to evolve spiking neural networks' structure and parameters for neuromorphic deployment. EONS determines simultaneously the number of neurons, number of synapses, connectivity, and parameters of the neurons and synapses over the course of evolution. EONS uses a direct, graph-based representation of the spiking neural network. Both crossover and mutation are used in EONS and both operate on the direct graph representation. With crossover, two parents are selected using tournament selection and two children

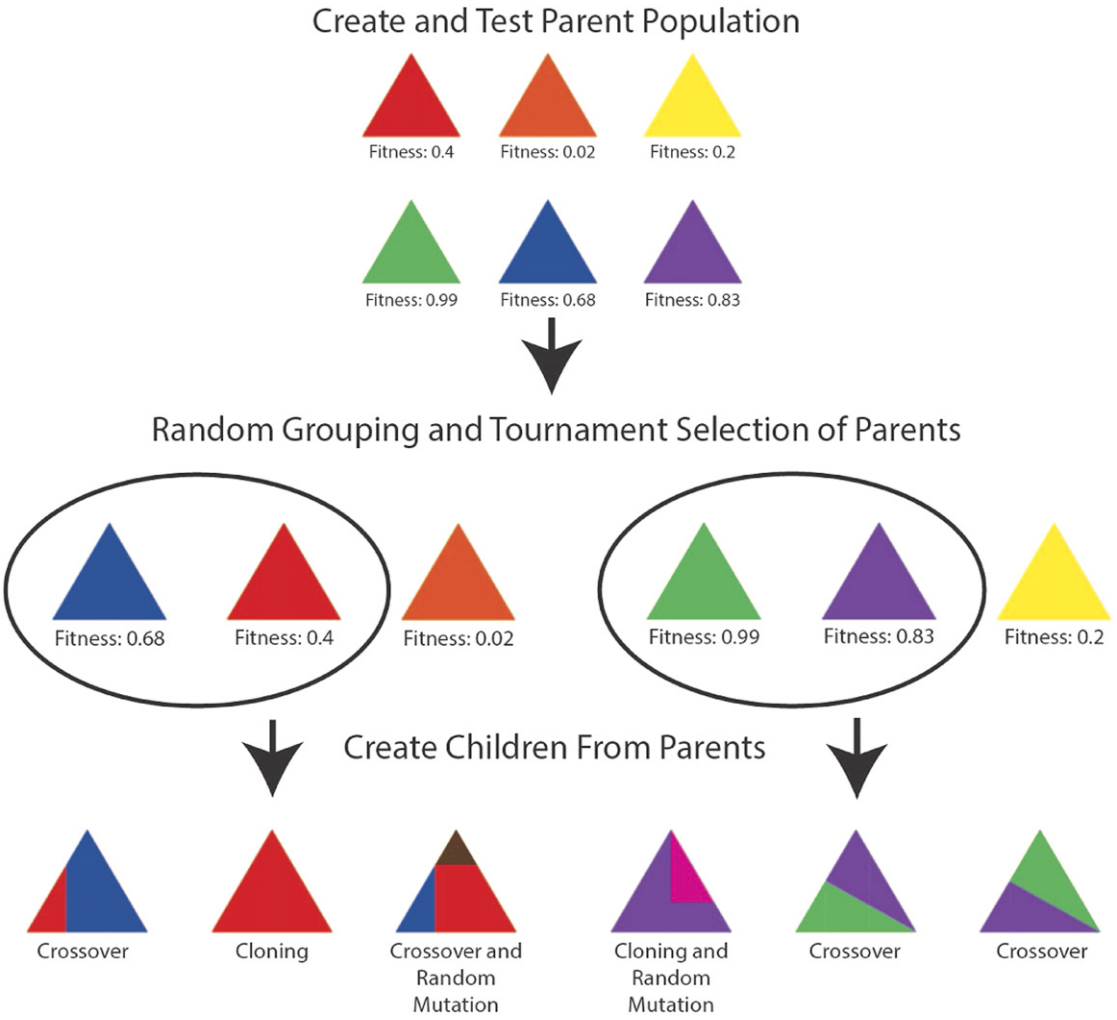


Figure 5. General workflow for an evolutionary algorithm.

are produced, each of which inherent characteristics from both parents. With random mutation, a small scale change is made to the parent network, such as adding or deleting a neuron or synapse or changing a small number (less than five) parameter values in the network. EONS parallelizes the fitness evaluation of each of the networks in the population across multiple compute cores. EONS uses a synchronous evolutionary approach, which means that the algorithm waits for all members of the population to complete before performing selection and reproduction operations. Because EONS directly represents the network, when evaluating the fitness function, we simply load the network specified by EONS directly onto the neuromorphic processor.

The second, library for evolutionary algorithms in Python (LEAP) [13] is an open-source Python library for evolutionary approaches. As LEAP does not currently support variable length genomes, we use LEAP to evolve the parameters (synaptic weights, synaptic delays, and neuronal thresholds) of a fixed structure network. We use a direct, real-valued representation to represent each of the parameters of the network. Because there are two parameters per synapse and one parameter per neuron, the length of the genome is $2n_s + n_n$, where n_s is the number of synapses and n_n is the number of neurons. With LEAP, we use an asynchronous parallel evolutionary approach, in which the compute resources used for fitness evaluation are constantly kept busy with networks to evaluate, i.e., there is no waiting for the entire population to finish before performing operations like selection and reproduction. We also use a newly introduced selection approach selection while evaluating [46], which is specifically designed for applications in which individuals that have higher performance also take longer to evaluate. Because LEAP uses a fixed network structure, there is a hyperparameter to determine the number of hidden neurons. The fixed network structure used is then composed of an input layer, a hidden layer, and an output layer, where all of the input neurons are connected to all of the hidden neurons and all of the hidden neurons are connected to all of the output neurons, but all of the hidden neurons are connected to each other as well.

For both EONS and LEAP, we used an input encoding approach to turn the LIDAR sensor values into spikes. The input encoding approach used is one that we found to perform well for control tasks such as autonomous

navigation in [41]. For each of the ten LIDAR values, we use two input neurons, each of which can spike at most eight times, and the value that they can spike into the neuron can be between 0 and 127 for μCaspian. As such, there are a total of 20 input neurons for each network. There is an output neuron for each of the 29 possible steering angles and each of the 11 possible speed values. The steering angle (similarly, speed) value is decided by which steering angle (speed) output neuron fires the most. For both evolutionary approaches, we evolve for a total of 5000 births. In the case of EONS, this equates to a population size of 100 for 50 generations.

3.4.2. Imitation learning

A very common machine learning approach for autonomous driving is imitation learning [12, 32, 54]. With imitation learning, an agent is trained to perform a task using 'observations' of another entity performing the task, specifically by collecting observations and actions as training data and labels respectively. In some cases, the entity that an agent is trained to mimic is a human. In our case, we collected an imitation learning dataset by using the waypoints of the training tracks and the waypoint following code provided in the F1TENTH simulator. We collected the observations (LIDAR sensors) and actions chosen by the waypoint follower (speed and steering angle) and saved those for our training dataset and labels. We can then frame this task as a classification task, where the input is the LIDAR sensor information and the output is either the speed value or the steering angle. For our imitation learning approaches, we train separate networks to predict the appropriate speed value and the appropriate angle value. We use this approach because the existing implementations of these frameworks implemented within the TENNLab framework only support a single classification label at a time.

We used two approaches for training spiking neural network classifiers for deployment to μCaspian that have already been implemented in the TENNLab framework and have been shown previously to perform well on classification tasks [43]. Both of these approaches require either non-neuromorphic post-processing or pre- and post-processing on the data. The first approach is a backpropagation-based training approach for spiking neural networks for neuromorphic deployment called Whetstone [47]. Whetstone trains neural networks with binary activation functions. It begins with a typical activation function like a sigmoid or rectified linear unit at the beginning of training and gradually 'sharpens' the activation functions to binary activations with thresholds of 0.5. Additionally, Whetstone uses n-hot output encoding to address the 'dead neuron' issue and a key to decode the output before passing it to a softmax function as post-processing. In this work, we restrict our attention to feed-forward networks with a single hidden layer with ten-hot output encoding. The approach we take for mapping Whetstone-trained networks to μCaspian is described in [43]. Because Whetstone operates directly on a fixed structure spiking neural network, we evaluated different numbers of hidden neurons in the hidden layer. We restricted our attention to a single hidden layer because in our initial experimental evaluations, multiple hidden layers resulted in poorer performance. As noted in [43], a quirk of Whetstone is that the input layer is not a binary activation, so we cannot encode the input layer directly into a spiking neural network. Thus, we perform a pre-processing step of calculating the first layer and feeding those values as input into the hidden layer. We also perform the output post-processing step of using the key to decode followed by the softmax evaluation, which is also not performed on the μCaspian system.

The second imitation learning approach we used is a simple reservoir computing approach [55]. In our reservoir computing approach, we use the same input encoding approach as in the evolutionary algorithms, so there are eight total input neurons. The hyperparameters of the reservoir approach are then the number of hidden neurons, the number of output neurons that go to the readout layer, and the probability of connectivity. We create a reservoir network with eight input neurons and the number of specified hidden neurons and output neurons. We use the probability of connectivity hyperparameter to then build a network where any of the neurons in the reservoir can be connected to each other with a synapse (including the inputs to inputs, output to inputs/hidden, outputs to outputs, etc). For each observation, we then feed the observation as input, simulate activity in the reservoir, and track the number of times each of the output neurons fires. We create a vector from the counts of each output neuron fires and feed that to a readout layer to give us the predicted value. In this case, we use the SGDClassifier with default parameters from scikit-learn [34] as the readout layer.

4. Results

We evaluate these algorithmic approaches and the best performing spiking neural networks that result from these approaches both in simulation and on the physical car on a physical racetrack. For each of the approaches, we will also show a plot describing the performance across all of the train and test tracks in simulation for all of the networks trained with that approach.

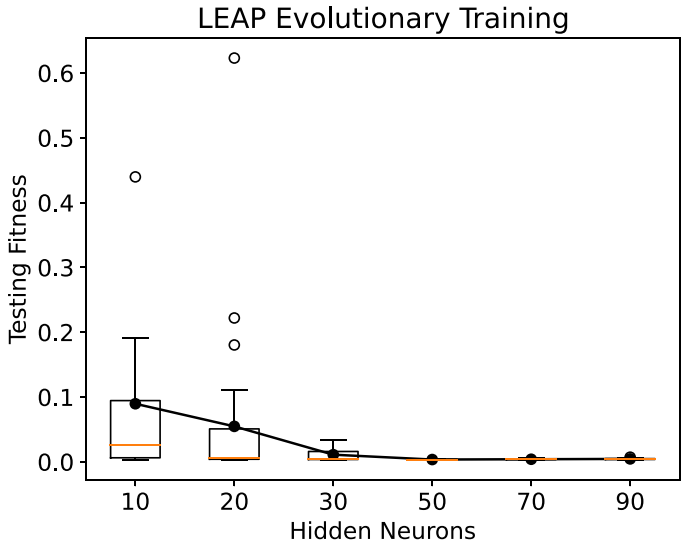


Figure 6. Effect of varying the number of neurons in the hidden layer for the LEAP-based evolutionary training approach. The mean for each number of hidden neurons is shown as black dots and connected with a line to show the trend.

4.1. Evolutionary learning results

Since the LEAP evolutionary approach does not currently optimize the network structure, we performed a hyperparameter grid search where the number of hidden neurons used in the network is the hyperparameter. We evaluated ten runs for each of the following numbers of hidden neurons: 10, 30, 50, 70, and 90. Because the best performing results occurred at 10, we further refined the number of hidden neurons to 20 and added this to our hyperparameter search. The testing results in simulation for this hyperparameter optimization approach are shown in figure 6. As can be seen in that figure, the best performance on average was still seen with ten hidden neurons; however, the best performance overall was seen with a network with 20 hidden neurons. The performance per training/testing map for all of the LEAP-trained networks are shown in figure 7. As can be seen in this plot, most of the networks trained with LEAP did not perform well. However, the best performing network trained with LEAP achieved near perfect or perfect performance on four out of five training tracks and eight out of 15 testing tracks.

Unlike LEAP, EONS learns both the parameters and structure of the spiking neural network simultaneously, reducing the hyperparameter optimization that is required for a given task. We evaluated a total of 30 different EONS runs for this task. The performance per training/testing map for the thirty EONS-trained networks are shown in figure 8. Unlike the LEAP trained networks, many of the EONS-trained networks performed well, indicating that performance can be significantly improved by evolving the structure of the network in addition to the parameters. As can be seen in figure 8, the best performing EONS-trained network performed well on four out of five of the training tracks and nine out of fifteen of the testing tracks. Unlike LEAP, however, there exists an EONS network that performs well on *each* of the training or testing tracks, but these results indicate that there are likely different classes of tracks, such that behaviors that perform well for some tracks do not perform well for others.

4.2. Imitation learning results

In the evolutionary approaches above, a single network was trained to produced both the speed and the steering angle. For imitation learning, we trained separate networks to predict the speed value and the angle. Both Whetstone and reservoir computing approaches have hyperparameters to define. Here, we used the same hyperparameter selection for both the speed network and the steering angle network; however, in the future, it may be worthwhile to expand the search space and allow for hyperparameters for each network.

For Whetstone, we performed a grid search over the number of neurons for the hidden layer, with five evaluations per number of hidden neurons. Figure 9 shows the results for different numbers of hidden neurons (10, 30, 50, 70, 90, 110, 130, 150, 170, and 190). As can be seen in these results, the best performing network only used ten hidden neurons in each network; however, there was also a local optima at 170 hidden neurons in the network. Figure 10 shows the results on each map for each Whetstone trained network. Interestingly, though the imitation learning dataset was drawn from the training tracks, the resulting networks did not necessarily perform well on the training tracks. In fact, in general, they tended to perform better (though still not especially well) on the testing tracks. The best performing network on the testing set in simulation did not actually perform well at all on the training tracks, but it was able to perform well on seven of the 15 testing tracks.

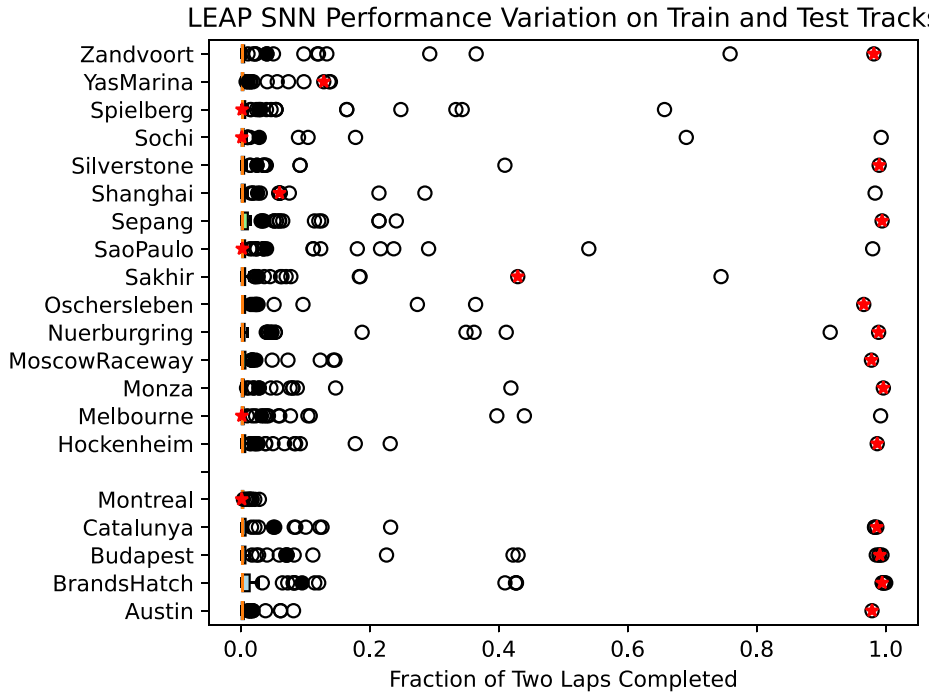


Figure 7. Performance across all SNNs evolved with LEAP across the train and test tracks in simulation. The best performing network based on average testing score is shown as a red star. The five training tracks are shown at the bottom, and the fifteen testing tracks are shown at the top.

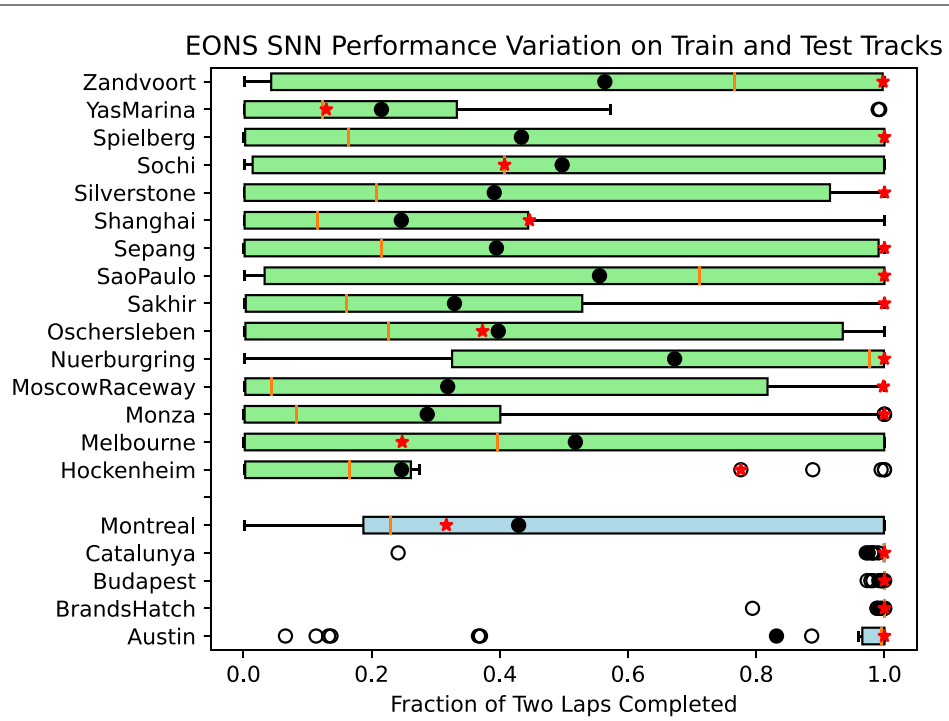


Figure 8. Performance across all SNNs evolved with EONS across the train and test tracks in simulation. The best performing network based on average testing score is shown as a red star. The five training tracks are shown at the bottom, and the fifteen testing tracks are shown at the top.

For reservoir computing, we also performed a hyperparameter grid search over the number of hidden neurons, as well as the probability of connectivity within the reservoir. The results for the hyperparameter search are given in figure 11, by hidden neurons, probability of connectivity, and the combination of the two. We evaluated ten reservoirs for each combination of hidden neurons and probability of connectivity. There was a clear best performing number of hidden neurons at 190. Although the best individual overall was found for probability of connectivity set to 0.3, it was not clear that this would generally be the case for this task.

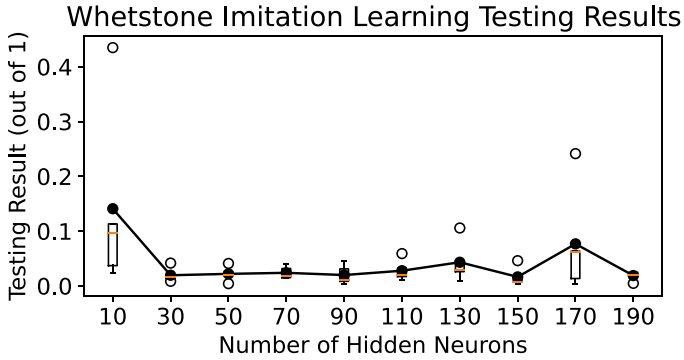


Figure 9. Effect of varying the number of neurons in the hidden layer for the Whetstone-based imitation learning approach. The mean values are plotted as black dots and connected with a black line to show trends in performance.

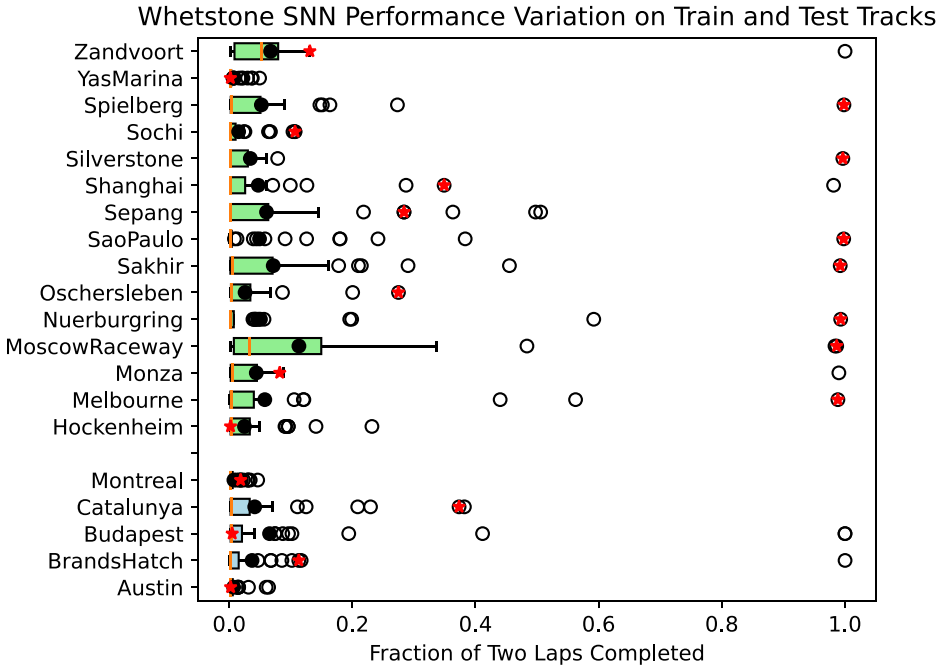


Figure 10. Performance across all SNNs trained with Whetstone across the train and test tracks in simulation. The best performing network based on average testing score is shown as a red star. The five training tracks are shown at the bottom, and the fifteen testing tracks are shown at the top.

Additionally, it may be worthwhile to investigate larger networks, as 190 hidden neurons was the largest number we investigated and achieved the best performance. We can also see from this figure, as well as from figure 12 (which again breaks down performance by map) that the reservoir computing approach is by far the worst across all training approaches for this application. The best performing network only performed reasonably well on one training track and one testing track.

4.3. Comparison between algorithmic approaches

In the previous sections, we have described the results across all four training approaches, two evolutionary and two imitation learning approaches with respect to accuracy. However, for edge deployment, accuracy is not the only concern. In table 2, we show the best performing networks for each training approach in terms of accuracy, number of neurons, number of synapses, and whether pre- or post-processing is required for that algorithm. As can be seen in this table, both the evolutionary approaches perform significantly better in terms of accuracy on this task. Additionally, the network sizes for the network generated with EONS and LEAP are significantly smaller than those produced by Whetstone and reservoir. Both Whetstone and reservoir require pre- and/or post-processing that is not implemented on the neuromorphic implementation, whereas neither LEAP nor EONS require such processing. As such, the evolutionary approaches are more well-suited to edge applications, where they can be implemented on smaller neuromorphic implementations that require

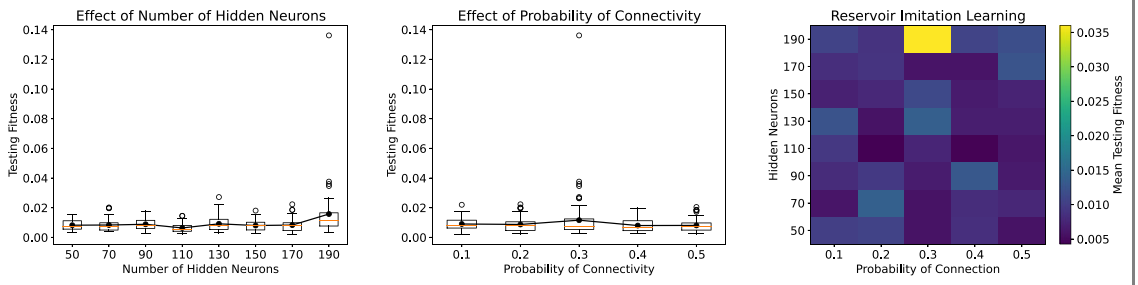


Figure 11. Reservoir computing imitation learning results, with varying number of hidden neurons and probability of connectivity, as well as the combination of both.

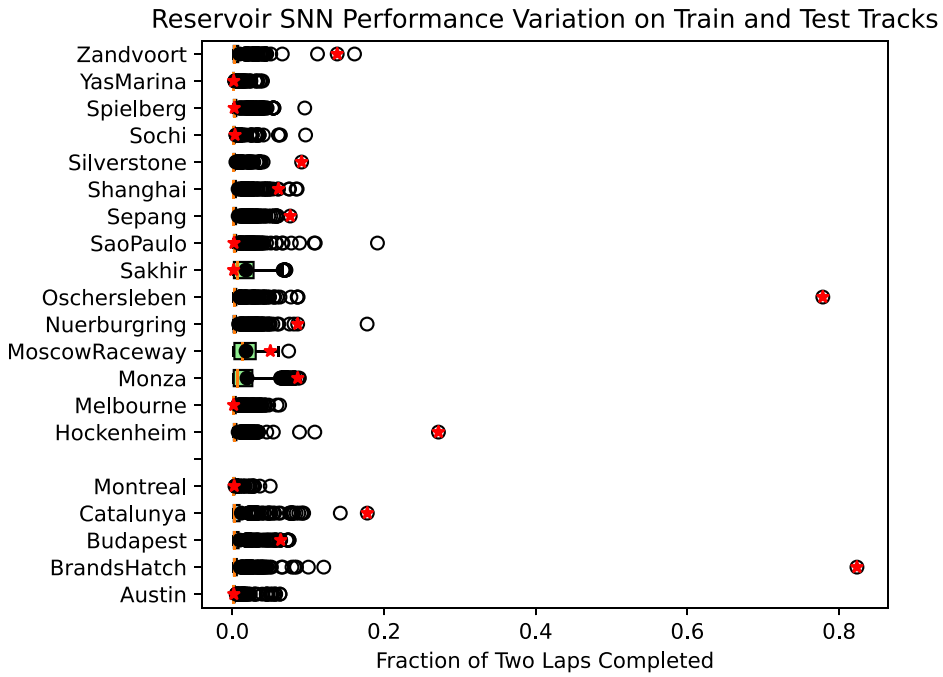


Figure 12. Performance across all SNNs trained with reservoir computing across the train and test tracks in simulation. The best performing network based on average testing score is shown as a red star. The five training tracks are shown at the bottom, and the fifteen testing tracks are shown at the top.

Table 2. Algorithm comparison.

	Algorithm	Testing accuracy	Neurons	Synapses	Pre-processing	Post-processing
Evolutionary	LEAP	0.623	80	2380	No	No
	EONS	0.785	60	97	No	No
Imitation learning	Whetstone	0.435	400	3800	Yes	Yes
	Reservoir	0.136	496	36 710	No	Yes

less power. Note that the best performing networks for Whetstone and reservoir are network sizes that will not fit on the μCaspian development board (which can hold 256 neurons and 4096 synapses). They would each require a larger FPGA with a larger Caspian architecture deployed to it, and thus would require more power. μCaspian is an event-driven architecture. Since smaller networks tend to have fewer events, the processing time required for smaller networks will be less, so overall energy usage should be less for a smaller network, even on the same FPGA. We did not directly measure power usage on the physical car in this case, as the power usage of μCaspian is negligible compared to the overall system power, but we intend to investigate this further in the future.

It is worth noting that the evolutionary approaches take significantly longer than the imitation learning approaches for training. In particular, the LEAP and EONS approaches take on the order of hours to complete

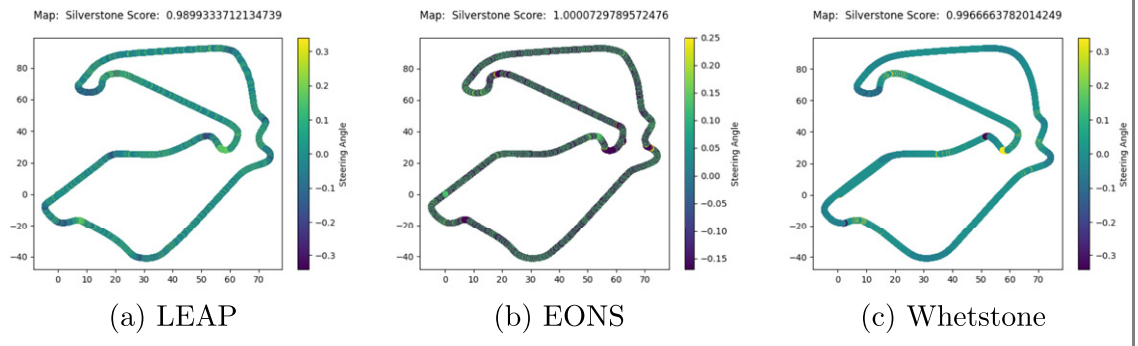


Figure 13. Different steering angle behaviors from each algorithm. These plots show how the best performing networks for LEAP, EONS, and Whetstone performed on the Silverstone track. We omit the reservoir computing results because they performed poorly on most of the tracks produced.

50 epochs of training, while the reservoir and Whetstone approaches take minutes to complete the training process. It is also worth noting that as better and better solutions are evolved with EONS or LEAP, the longer each epoch takes. This is because networks that perform well take longer to evaluate. If training time is of significant concern in a control application, then imitation learning may be a better approach.

Figure 13 shows how the best networks for LEAP, EONS, and Whetstone perform in terms of steering angle output when driving along the Silverstone track. Note that we omit the reservoir computing results because they performed poorly on most tracks. From these results, we can note a few interesting things about the control strategies that are developed for each approach. For the LEAP trained network, the steering angle oscillates throughout the course, but its oscillations tend to be between small positive and negative values, with larger steering angle values (in magnitude) when encountering turns. For EONS, the steering angle values perform lots of oscillations between relatively large positive and negative values throughout driving along the track. This is consistent with 'weaving' behavior that we see when we physically drive the car with many EONS networks. Though the objective function that we define attempts to eliminate oscillating values such as these, it does not reward that behavior until the network has already learned to complete two laps; thus, we may not be training long enough to evolve out weaving behavior for either EONS or LEAP.

For the Whetstone trained network, however, the steering angle values are much more constant and only update when encountering a turn. This is consistent with what we would expect from an imitation learning approach, because the agent that the network was trained to imitate drives straight along the path until encountering turns. It is interesting to note, however, that the networks that are evolved tended to outperform these imitation learning approaches. As such, there may be some benefit to 'weaving' behaviors, perhaps because the network is constantly shifting positions and obtaining more information from the sensors about the track layout.

An observation from the summaries of results across the different approaches (figures 7, 8, 10 and 12) is that there are clearly more difficult tracks and less difficult tracks. However, we do note that though there is not a single network that performs well on every track, networks were evolved that performed well on every individual track. Unsurprisingly, this tells us that a single control strategy may not be appropriate for all possible tracks. To probe further into this phenomena, we examined which of the tracks each of the 30 EONS-trained networks completed (shown in figure 14). In this figure, we can see that the three most difficult tracks are Shanghai, which only had two networks that completed it, and YasMarina and Hockenheim, both of which only had three that completed them. Those three tracks are shown in figure 15. We can see that each of these tracks has very sharp turns, which are where most of the trained networks fail. To address this issue, it may be worthwhile to include more tracks with very sharp turns in our training set.

4.4. Real-world evaluation

To evaluate the trained networks in the real-world, we constructed a racetrack with walls made of cardboard. The length of our real-world track was 123 feet. We evaluated the top ten performing networks (in terms of testing score) across all algorithms. Nine of those networks were produced using EONS and one was produced using LEAP. We then measured each of those networks' performance on the track three times from three roughly equidistant starting points along the track. A portion of the track itself, as well as the results are shown in figure 16. The real world track layout does not match the layout of any of the training or testing tracks.

As can be seen in the figure, there was a significant drop in performance from expected performance in testing to real-world performance on the car. However, the trends in performance in the real-world roughly

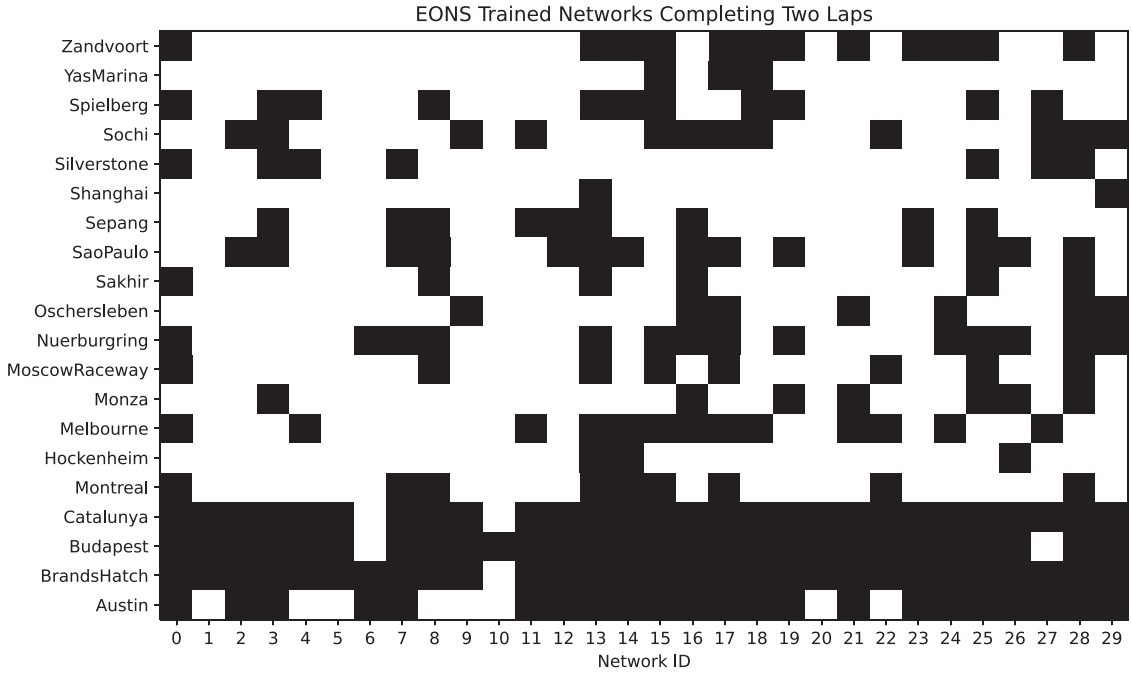
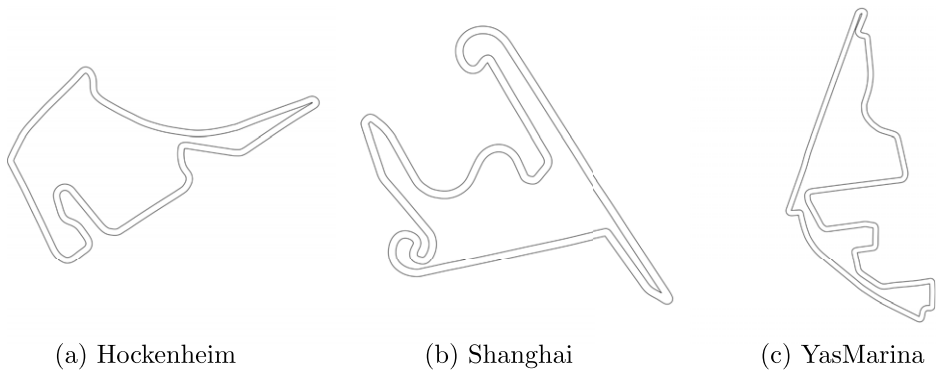
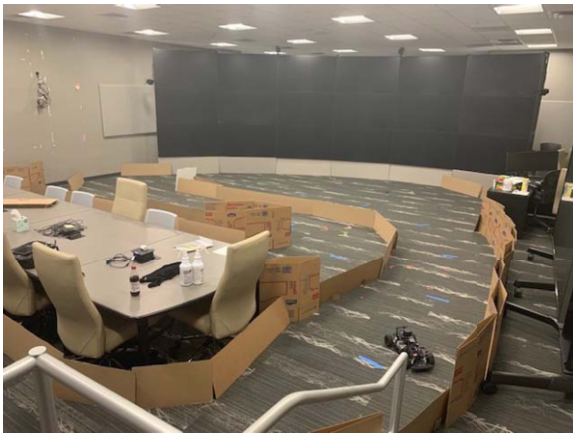


Figure 14. Networks that completed two laps on each track are shaded in black.



(a) Hockenheim (b) Shanghai (c) YasMarina

Figure 15. Formula One tracks that were the most difficult to train for with EONS [31].



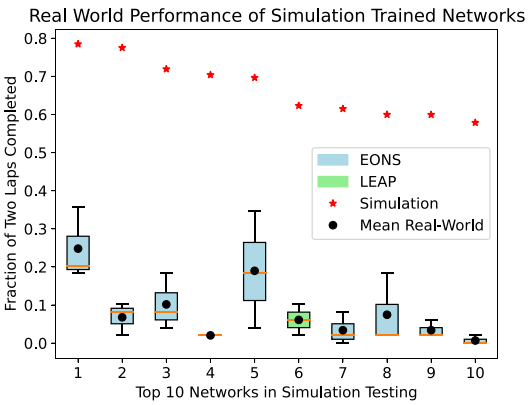


Figure 16. A portion of the real world track (left), and results collected on the real world track for the top ten performing networks (right).

tracked with the testing performance, insofar as if the network had better performance in testing in simulation, it tended to perform better in the real-world (though not always).

As for why there was such a significant drop in performance from simulation to real-world, in our experimentation with the physical car, we found that the LIDAR system used on the physical car was extremely sensitive to the materials used to build the wall, as well as any small gaps or deformations in the wall. The simulator used in training and testing assumes perfect LIDAR and perfect track walls. For future training, we intend to investigate adding noise into the simulation of the LIDAR sensing to better approximate the behavior expected in the real-world. Additionally, replicating a real-world track that matches the characteristics of a Formula 1 racetrack is non-trivial. In constructing the track, we may be producing conditions (turning angles, etc) that are not present at all in our training set. To address this issue, in the future, we intend to create synthetic tracks to train on along with the real-world tracks.

5. Discussion

In this work, we describe an application for neuromorphic control at the edge, and we evaluate several algorithmic approaches for addressing this challenge. In particular, the application is an autonomous racing example with a physical hardware platform, where the hardware specifications and the software are freely available to the community. In our study of neuromorphic algorithms for control at the edge, we evaluated two evolutionary approaches and two imitation learning approaches. Our key observations are as follows:

Observation 1: evolutionary approaches tended to outperform the imitation learning approaches in terms of accuracy. There are several reasons why this may be the case. The imitation learning approaches followed data collected by a waypoint follower. This may not be the best agent to imitate or to collect an imitation learning dataset from; however, collecting imitation learning datasets such as these may be difficult. Another point is that we separated the prediction of speed and the prediction of steering angle networks. Future approaches may train networks that predict both simultaneously.

Observation 2: imitation learning approaches train faster than evolutionary approaches, but they also require a dataset from which to train. The imitation learning approaches train significantly faster (in minutes rather than hours) because they are not interacting directly with the simulated environment. However, this assumes the existence of an imitation learning dataset, which may be costly to obtain. Evolutionary approaches require a simulated environment on which to train. As tasks and thus the simulated environments complexify, simulating every individual in an evolutionary approach may take prohibitively long times.

Observation 3: evolving structure improves performance. We evaluated two evolutionary approaches, LEAP and EONS. Our LEAP approach does not currently evolve the network's structure (number of neurons and number of synapses). For a fair comparison, we evaluated a variety of structural hyperparameters, but the results still did not outperform EONS, which evolves the network structure simultaneously with the weights.

Observation 4: evolutionary approaches that evolve the structure result in significantly smaller networks than other approaches. The networks evolved using EONS were significantly smaller in terms of both neurons and synapses than other approaches. LEAP produced networks that were comparable in terms of the number of neurons, but because the number of synapses was fixed and fully connected in the hidden layers, LEAP required significantly more synapses. Smaller networks are especially important for deployment at the edge.

6. Conclusion

There is great opportunity to use neuromorphic computing for edge applications broadly and specifically for control applications at the edge. In this work, we evaluate training approaches for neuromorphic control at the edge for an autonomous racing platform. We found that evolutionary approaches tended to produce both more accurate networks, as well as smaller networks that were more well-suited for edge deployment than imitation learning approaches.

In the future, we plan to evaluate other approaches for neuromorphic training on this application, including reinforcement learning-based approaches. We also hope to gradually increase the difficulty of the task to further evaluate algorithmic approaches, including adding obstacles to the tracks as well as other racing vehicles. This work provides one step in the direction of autonomous vehicle control using neuromorphic computing, and it demonstrates that existing neuromorphic algorithms can produce solutions that perform well on this task and that can be feasibly deployed to edge neuromorphic implementations.

Additionally, the complete workflow that we introduced in this task (shown in figure 1) is enabled by the TENNLab neuromorphic software framework and thus allows for interchangeable neuromorphic hardware

implementations as well as interchangeable applications. In the future, we plan to investigate the performance of other neuromorphic hardware implementations, as well as other control applications at the edge.

Acknowledgments

This material is based upon work supported by the U.S. Department of Energy, Office of Science, Office of Advanced Scientific Computing Research, under Contract number DE-AC05-00OR22725.

Data availability statement

The data that support the findings of this study are available upon reasonable request from the authors.

References

[1] Akhyar S and Omatu S 1993 Neuromorphic self-tuning PID controller *IEEE Int. Conf. on Neural Networks* (IEEE) pp 552–7
[2] Ambrose J D, Foshie A Z, Dean M E, Plank J S, Rose G S, Mitchell J P, Schuman C D and Bruer G 2020 Grant: ground-roaming autonomous neuromorphic targeter *2020 Int. Joint Conf. on Neural Networks (IJCNN)* (IEEE) pp 1–8
[3] Amravati A, Nasir S B, Thangadurai S, Yoon I and Raychowdhury A 2018 A 55 nm time-domain mixed-signal neuromorphic accelerator with stochastic synapses and embedded reinforcement learning for autonomous micro-robots *2018 IEEE Int. Solid-State Circuits Conf.-(ISSCC)* (IEEE) pp 124–6
[4] Babu V S and Behl M 2020 f1tenth.dev-an open-source ROS based f1/10 autonomous racing simulator *2020 IEEE 16th Int. Conf. on Automation Science and Engineering (CASE)* (IEEE) pp 1614–20
[5] Bauer F C, Muir D R and Indiveri G 2019 Real-time ultra-low power ECG anomaly detection using an event-driven neuromorphic processor *IEEE Trans. Biomed. Circuits Syst.* **13** 1575–82
[6] Baxter J A, Merced D A, Costinett D J, Tolbert L M and Ozpineci B 2018 Review of electrical architectures and power requirements for automated vehicles *2018 IEEE Transportation Electrification Conf. and Expo (ITEC)* (IEEE) pp 944–9
[7] Peter B, Choo X, Hunsberger E and Eliasmith C 2019 Benchmarking keyword spotting efficiency on neuromorphic hardware *Proc. of the 7th Annual Neuro-Inspired Computational Elements Workshop* pp 1–8
[8] Cass S 2020 Nvidia makes it easy to embed AI: the Jetson Nano packs a lot of machine-learning power into DIY projects—(hands on) *IEEE Spectr.* **57** 14–6
[9] Chang X, Li W, Xia C, Ma J, Cao J, Khan S U and Zomaya A Y 2018 From insight to impact: building a sustainable edge computing platform for smart homes *2018 IEEE 24th Int. Conf. on Parallel and Distributed Systems (ICPADS)* (IEEE) pp 928–36
[10] Chen B *et al* 2020 A memristor-based hybrid analog-digital computing platform for mobile robotics *Sci. Robot.* **5** eabb6938
[11] Chen G, Cao H, Conradt J, Tang H, Rohrbein F and Knoll A 2020 Event-based neuromorphic vision for autonomous driving: a paradigm shift for bio-inspired visual sensing and perception *IEEE Signal Process. Mag.* **37** 34–49
[12] Chen J, Yuan B and Tomizuka M 2019 Deep imitation learning for autonomous driving in generic urban scenarios with enhanced safety *2019 IEEE/RSJ Int. Conf. on Intelligent Robots and Systems (IROS)* (IEEE) pp 2884–90
[13] Coletti M A, Scott E O and Bassett J K 2020 Library for evolutionary algorithms in python (LEAP) *Proc. of the 2020 Genetic and Evolutionary Computation Conf. Companion* pp 1571–9
[14] Davies M, Wild A, Orchard G, Sandamirskaya Y, Guerra G A F, Joshi P, Plank P and Risbud S R 2021 Advancing neuromorphic computing with Loihi: a survey of results and outlook *Proc. IEEE* **109** 911–34
[15] Farag W 2020 Complex trajectory tracking using PID control for autonomous driving *Int. J. Intell. Transp. Syst. Res.* **18** 356–66
[16] Feng C, Wang Y, Chen Q, Strbac G and Kang C 2021 Smart grid encounters edge computing: opportunities and applications *Adv. Appl. Energy* **1** 100006
[17] Fischl K D *et al* 2017 Neuromorphic self-driving robot with retinomorphic vision and spike-based processing/closed-loop control *2017 51st Annual Conf. on Information Sciences and Systems (CISS)* (IEEE) pp 1–6
[18] Gawron J H, Keoleian G A, De Kleine R D, Wallington T J and Kim H C 2018 Life cycle assessment of connected and automated vehicles: sensing and computing subsystem and vehicle level effects *Environ. Sci. Technol.* **52** 3249–56
[19] Glatz S, Martel J, Kreiser R, Qiao N and Sandamirskaya Y 2019 Adaptive motor control and learning in a spiking neural network realised on a mixed-signal neuromorphic processor *2019 Int. Conf. on Robotics and Automation (ICRA)* (IEEE) pp 9631–7
[20] Habu Y, Uta K and Fukuoka Y 2019 Three-dimensional walking of a simulated muscle-driven quadruped robot with neuromorphic two-level central pattern generators *Int. J. Adv. Robot. Syst.* **16** 1729881419885288
[21] Hagenaars J J, Paredes-Vallés F, Bohté S M and De Croon G C H E 2020 Evolved neuromorphic control for high speed divergence-based landings of MAVs *IEEE Robot. Autom. Lett.* **5** 6239–46
[22] Hwu T, Isbell J, Oros N and Krichmar J 2017 A self-driving robot using deep convolutional neural networks on neuromorphic hardware *2017 Int. Joint Conf. on Neural Networks (IJCNN)* (IEEE) pp 635–41
[23] Jain A, O'Kelly M, Chaudhari P and Morari M 2020 BayesRace: learning to race autonomously using prior experience (arXiv:2005.04755)
[24] James C D *et al* 2017 A historical survey of algorithms and hardware architectures for neural-inspired and neuromorphic computing applications *Biol. Insp. Cogn. Archit.* **19** 49–64
[25] Li B, Cao H, Qu Z, Hu Y, Wang Z and Liang Z 2020 Event-based robotic grasping detection with neuromorphic vision sensor and event-grasping dataset *Front. Neurorobot.* **14** 51
[26] Liang X, Liu Y, Chen T, Liu M and Yang Q 2019 Federated transfer reinforcement learning for autonomous driving (arXiv:1910.06001)
[27] Mirus F, Zorn B and Conradt J 2019 Short-term trajectory planning using reinforcement learning within a neuromorphic control architecture *ESANN*
[28] Mitchell J P, Bruer G, Dean M E, Plank J S, Rose G S and Schuman C D 2017 Neon: neuromorphic control for autonomous robotic navigation *2017 IEEE Int. Symp. on Robotics and Intelligent Sensors (IRIS)* (IEEE) pp 136–42

[29] Mitchell J P, Schuman C D, Patton R M and Potok T E 2020 Caspian: a neuromorphic development platform *Proc. of the Neuro-Inspired Computational Elements Workshop* pp 1–6
[30] Mitchell J P, Schuman C D and Potok T E 2020 A small, low cost event-driven architecture for spiking neural networks on FPGAs *Int. Conf. on Neuromorphic Systems 2020* pp 1–4
[31] O'Kelly M, Zheng H, Karthik D and Mangharam R 2020 F1TENTH: an open-source evaluation environment for continuous control and reinforcement learning *NeurIPS 2019 Competition and Demonstration Track* (PMLR) pp 77–89
[32] Pan Y, Cheng C-A, Saigol K, Lee K, Yan X, Theodorou E A and Boots B 2020 Imitation learning for agile autonomous driving *Int. J. Robot. Res.* **39** 286–302
[33] Patton R *et al* 2021 Neuromorphic computing for autonomous racing *Proc. of the Int. Conf. on Neuromorphic Systems (ICONS) 2021* (ACM)
[34] Pedregosa F *et al* 2011 Scikit-learn: machine learning in python *J. Mach. Learn. Res.* **12** 2825–30
[35] Piñero-Fuentes E, Canas-Moreno S, Rios-Navarro A, Delbruck T and Linares-Barranco A 2021 Autonomous driving of a rover-like robot using neuromorphic computing *Int. Work-Conf. on Artificial Neural Networks* (Springer) pp 57–68
[36] Plank J *et al* 2019 The TENNLab suite of LIDAR-based control applications for recurrent, spiking, neuromorphic systems *Technical Report* Oak Ridge National Lab.(ORNL) Oak Ridge, TN, United States
[37] Plank J S, Schuman C D, Bruer G, Dean M E and Rose G S 2018 The TENNLab exploratory neuromorphic computing framework *IEEE Lett. Comput. Soc.* **1** 17–20
[38] Polykretis I, Tang G and Michmizos K P 2020 An astrocyte-modulated neuromorphic central pattern generator for hexapod robot locomotion on Intel's Loihi *Int. Conf. on Neuromorphic Systems 2020* pp 1–9
[39] Rosenfeld B, Simeone O and Rajendran B 2019 Learning first-to-spike policies for neuromorphic control using policy gradients *2019 IEEE 20th Int. Workshop on Signal Processing Advances in Wireless Communications (SPAWC)* (IEEE) pp 1–5
[40] Schuman C D, Mitchell J P, Patton R M, Potok T E and Plank J S 2020 Evolutionary optimization for neuromorphic systems *Proc. of the Neuro-Inspired Computational Elements Workshop* pp 1–9
[41] Schuman C D, Plank J S, Bruer G and Anantharaj J 2019 Non-traditional input encoding schemes for spiking neuromorphic systems *2019 Int. Joint Conf. on Neural Networks (IJCNN)* (IEEE) pp 1–10
[42] Schuman C D, Plank J S, Disney A and Reynolds J 2016 An evolutionary optimization framework for neural networks and neuromorphic architectures *2016 Int. Joint Conf. on Neural Networks (IJCNN)* (IEEE) pp 145–54
[43] Schuman C D, Plank J S, Parsa M, Kulkarni S R, Skuda N and Mitchell J P 2021 A software framework for comparing training approaches for spiking neuromorphic systems *2021 Int. Joint Conf. on Neural Networks (IJCNN)* pp 1–10
[44] Schuman C D, Potok T E, Patton R M, Birdwell J D, Dean M E, Rose G S and James S P 2017 A survey of neuromorphic computing and neural networks in hardware (arXiv:1705.06963)
[45] Schuman C D, Young S R, Mitchell J P, Johnston J T, Rose D, Maldonado B P and Kaul B C 2020 Low size, weight, and power neuromorphic computing to improve combustion engine efficiency *2020 11th Int. Green and Sustainable Computing Workshops (IGSC)* (IEEE) pp 1–8
[46] Scott E, Coletti M, Schuman C, Kay B, Kulkarni S, Parsa M and De Jong K 2021 Avoiding excess computation in asynchronous evolutionary algorithms *Proc. of United Kingdom Computational Intelligence (UKCI) Workshop 2021*
[47] Severa W, Vineyard C M, Dellana R, Verzi S J and Aimone J B 2019 Training deep neural networks for binary communication with the Whetstone method *Nat. Mach. Intell.* **1** 86–94
[48] Shalumov A, Halaly R and Ezra Tsur E 2021 LIDAR-driven spiking neural network for collision avoidance in autonomous driving *Bioinsp. Biomim.* **16** 066016
[49] Shetty C, Nitchith S, Rawat R, Nandakumar S R, Shah P, Kulkarni S and Rajendran B 2015 Live demonstration: spiking neural circuit based navigation inspired by *C. elegans* thermotaxis *2015 IEEE Int. Symp. on Circuits and Systems (ISCAS)* (IEEE) p 1905
[50] Sinha A, O'Kelly M, Zheng H, Mangharam R, Duchi J and Tedrake R 2020 Formulazero: distributionally robust online adaptation via offline population synthesis *Int. Conf. on Machine Learning* (PMLR) pp 8992–9004
[51] Spaeth A, Tebyani M, Haussler D and Teodorescu M 2020 Neuromorphic closed-loop control of a flexible modular robot by a simulated spiking central pattern generator *2020 3rd IEEE Int. Conf. on Soft Robotics (RoboSoft)* (IEEE) pp 46–51
[52] Stagsted R, Vitale A, Binz J, Renner A, Larsen L B and Sandamirskaya Y 2020 Towards neuromorphic control: a spiking neural network based PID controller for UAV *Robotics: Sci. Syst.*
[53] Stewart T C 2012 *A Technical Overview of the Neural Engineering Framework* vol 110 (University of Waterloo)
[54] Sun L, Peng C, Zhan W and Tomizuka M 2018 A fast integrated planning and control framework for autonomous driving via imitation learning *Dynamic Systems and Control Conf.* vol 51913 (American Society of Mechanical Engineers) p V003T37A012
[55] Tanaka G, Yamane T, Héroux J B, Nakane R, Kanazawa N, Takeda S, Numata H, Nakano D and Hirose A 2019 Recent advances in physical reservoir computing: a review *Neural Netw.* **115** 100–23
[56] Viale A, Marchisio A, Martina M, Masera G and Shafique M 2021 CarSNN: an efficient spiking neural network for event-based autonomous cars on the Loihi neuromorphic research processor *2021 Int. Joint Conf. on Neural Networks (IJCNN)* (IEEE) pp 1–10
[57] Wang C, Yang Z, Wang S, Wang P, Wang C-Y, Pan C, Cheng B, Liang S-J and Miao F 2020 A Braitenberg vehicle based on memristive neuromorphic circuits *Adv. Intell. Syst.* **2** 1900103
[58] Wunderlich T *et al* 2019 Demonstrating advantages of neuromorphic computation: a pilot study *Front. Neurosci.* **13** 260
[59] Yao B and Jiang C 2010 Advanced motion control: from classical PID to nonlinear adaptive robust control *2010 11th IEEE Int. Workshop on Advanced Motion Control (AMC)* (IEEE) pp 815–29
[60] Zaidel Y, Shalumov A, Volinski A, Supic L and Ezra Tsur E 2021 Neuromorphic NEF-based inverse kinematics and PID control *Front. Neurorobot.* **15** 631159
[61] Zhao J, Donati E and Indiveri G 2020 Neuromorphic implementation of spiking relational neural network for motor control *2020 2nd IEEE Int. Conf. on Artificial Intelligence Circuits and Systems (AICAS)* (IEEE) pp 89–93
[62] Zhao J, Risi N, Monforte M, Bartolozzi C, Indiveri G and Donati E 2020 Closed-loop spiking control on a neuromorphic processor implemented on the iCub *IEEE J. Emerg. Sel. Top. Circuits Syst.* **10** 546–56